ALPHA OMEGA INTEGRATION LLC

Technical Description: AI Agents for the Army Accessions Information Environment (AIE)

Agentic automation for accessions case management and proactive documentation

---

Prepared for: AIE technical evaluation (architecture, feasibility, and governance review)

Prepared by: Alpha Omega — National Security Solutions

Status: Reference architecture, demonstrated live in a commercial Salesforce trial org

Classification: Unclassified / pre-decisional draft for technical review

Contents

1. Purpose and scope

This document describes the software agents Alpha Omega built to automate accessions case management within an environment modeled on the Army Accessions Information Environment (AIE). It explains what each agent does, how the agents were created, the infrastructure they run on, and the specific role they play across the recruiting and accessions lifecycle. It is written for AIE technical evaluators assessing feasibility, architecture, and governance.

The central design goal is cycle-time reduction. Accessions timelines are dominated not by the forms themselves but by the iterative request-wait-resubmit loop around medical documentation, waivers, and security screening. The agents described here compress that loop by predicting and assembling the required documentation package the moment a case is created, drafting the applicant outreach for human approval, and instrumenting every action so cycle time becomes measurable data rather than anecdote.

Scope note. The capability was demonstrated live in a commercial Salesforce trial edition. All cadet names and data in this document and the demonstration are fictional placeholders; no real applicant PII was used. Production deployment to an IL4/FedRAMP-authorized environment, a governed model boundary, and licensed Agentforce/Einstein services are addressed in Sections 7 and 10 and flagged as assumptions in Section 11.

2. Architecture overview

The solution uses a **hybrid agentic architecture** organized in three cooperating layers. The layering is deliberate: it lets the same business outcome — a complete documentation package, an AI case summary, and a recruiter task — be produced in any Salesforce edition, so the capability runs identically whether or not licensed AI services are present.

2.1 Native automation layer

A record-triggered Salesforce Flow runs on the platform itself. When a Cadet Case is saved with a qualifying medical factor, the Flow synchronously creates the first predicted Document Request and sets case fields. This layer requires no external compute and runs inside the org's trust boundary.

2.2 External agent layer

A small roster of cooperating external agents reaches into Salesforce through its REST API to complete work the platform Flow or the current edition cannot: finishing the full policy-driven document checklist for any medical factor, generating an AI case summary and drafted applicant request, and queuing a recruiter follow-up task. The agents are idempotent, so they never duplicate what the native Flow already created. They can run headless on a schedule or, where API credentials are not available, through a supervised browser-automation path.

2.3 Licensed AI layer (drop-in)

In a fully licensed org, the external Copilot agent is replaced by native Einstein prompt templates and an Agentforce assistant. The orchestration and the data model are unchanged; only the component that produces the natural-language summary moves inside the platform. This makes the external agent layer a portable stand-in rather than a permanent dependency.

2.4 Process overview (Figure 1)

Figure 1 summarizes the end-to-end process on a single page. The top band shows the **core automation loop**: one input — a Cadet Case saved with a medical factor — triggers the native Flow to create the first predicted document, after which the external agents complete the full policy-driven checklist, draft the case summary and applicant outreach, and queue the recruiter task, leaving a connected, instrumented case whose value meter ticks up by one cycle eliminated for each document produced. The agents act against the factor-to-document policy table, so adapting to a policy change is a data edit rather than a code change.

The middle band is the **SEAD 4 human-in-the-loop guardrail** that governs the entire flow: AI surfaces, drafts, and automates, but a human approves every applicant send and owns every eligibility, suitability, and clearance decision. The lower band maps the agents onto the **ten-stage accessions lifecycle**, color-coded by delivery layer — native (live), external agent (live), and licensed (Agentforce/Einstein) — with the corresponding AIE RTM identifiers and the medical/waiver bottleneck (Stage 3) marked as the stage demonstrated live. The footer lists the infrastructure the capability runs on, with the IL4/FedRAMP posture shown as target state.

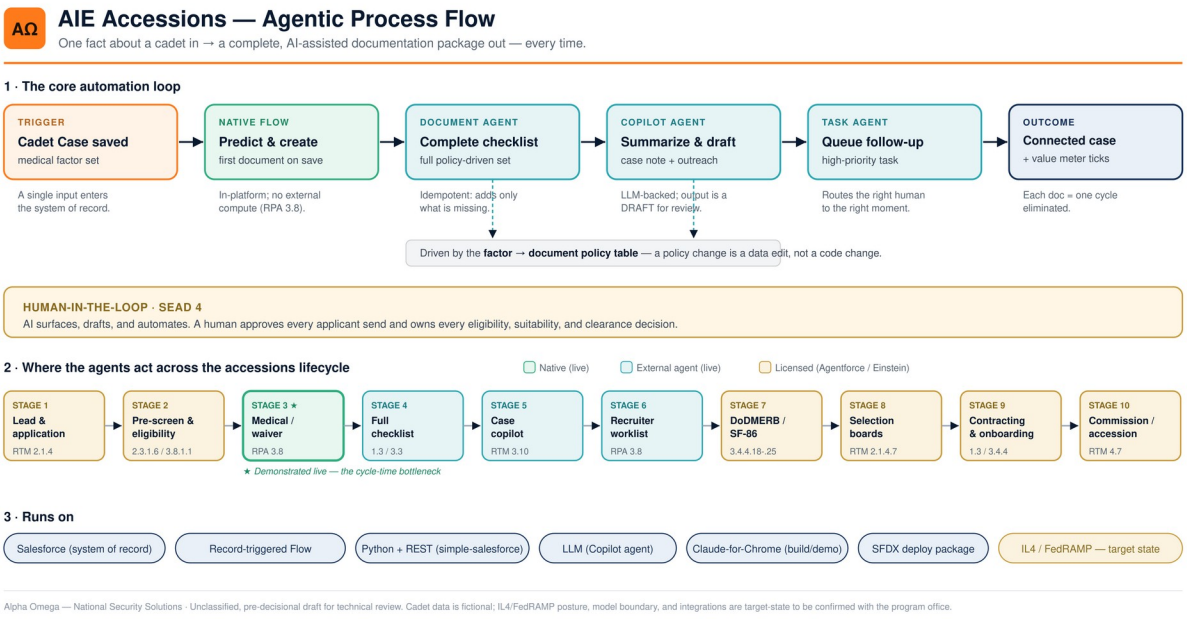


Figure 1. AIE accessions agentic process flow — core automation loop, SEAD 4 guardrail, lifecycle mapping, and infrastructure.

3. The agent roster

The external layer is implemented as a set of single-responsibility agents coordinated by an orchestrator. Separating responsibilities keeps each agent small, independently testable, and safe to re-run.

Agent	Responsibility	Salesforce action
Orchestrator	Sequences the run and prints the value summary (the cycle-time meter)	Invokes the agents below in order
SeedAgent	Creates the demonstration cadet roster across varied medical factors	Inserts Cadet Cases; setting the medical factor fires the native Flow
DocumentAgent	Completes the full, policy-driven document checklist per factor; idempotent	Creates only the missing Document Requests (predicted, requested)
CopilotAgent	Einstein replacement: case summary, gap check, and drafted applicant request	Writes a Note on the cadet for human approval before any send
TaskAgent	The administrative agent: prepares the recruiter follow-up	Creates a high-priority Task on the cadet
ResetAgent	Tears down prior demonstration data so each run starts clean	Deletes Document Requests, then Cadet Cases

Each agent has a narrow contract. The DocumentAgent, for example, reads a cadet's existing documents, computes the difference against the required set for that medical factor, and creates only what is missing — so a partial run, a retry, or a native-Flow head start all converge to the same complete state.

4. How the agents were created

4.1 Design method

The agents were derived directly from the accessions documentation policy. The core artifact is a **policy table** mapping each medical factor to the exact set of documents the government will require (Appendix A). This table — not hard-coded branching logic — is the single source of truth, so adapting to a policy change is a data edit rather than a development cycle. The DocumentAgent reads the table; the native Flow encodes the most common path for synchronous, in-platform creation.

4.2 The Copilot (LLM-backed) agent

The CopilotAgent composes a short case summary and a courteous draft applicant message from the case's structured fields. It is prompt-driven against a large language model through the Anthropic API, with an explicit instruction set: summarize path/branch, cycle stage, SF-86 status, days to the 60-day SECRET window, and the single next best action; produce the applicant message clearly marked as a draft; and make no eligibility, suitability, or clearance determination. If the model is unavailable, the agent falls back to deterministic text so a run never fails. In a licensed deployment this component is replaced by an Einstein prompt template and an Agentforce assistant operating within the platform's governed model boundary.

4.3 Idempotency and safety by construction

Every write is guarded. The DocumentAgent creates only missing documents; the orchestrator can be run with a dry-run flag that reports intended actions without writing; the ResetAgent makes runs repeatable. No agent performs a destructive or irreversible action without an explicit operator instruction, and no agent sends anything to an applicant — it only drafts.

4.4 How the build itself was performed

The Salesforce configuration (custom objects, fields, the record-triggered Flow, list views, and sample records) was built live through a supervised browser-automation agent (Claude for Chrome) driving the Salesforce Setup and record UIs, pausing for human confirmation before each meaningful change. The same browser path can run the demonstration end to end in any edition. The headless Python agents and the platform metadata were then captured into a deploy-ready SFDX package for repeatable provisioning.

5. Infrastructure and technology stack

Component	Description
Platform of record	Salesforce (Lightning). Custom objects Cadet Case (parent) and Document Request (child), related by lookup, with fields for Medical Factor, SF-86 Status, Cycle Stage, Contract Date, Predicted, Status, and Due Date.
Native automation	Record-triggered Flow (run-after-save) that creates the first predicted Document Request and sets case fields on cadet creation.
Agent runtime	Python. Salesforce access via the simple-salesforce REST client; case notes written via the ContentNote / ContentDocumentLink REST endpoints; tasks via the Task object.
Language model	Anthropic Claude via the Anthropic API for the Copilot summaries/drafts, with a deterministic offline fallback. In production this is replaced by an in-boundary, governed model (Einstein / Agentforce).
Supervised browser agent	Claude for Chrome automation for live, credential-free operation in any edition and for the live demonstration; performs the identical UI steps a human would.
Packaging & deployment	Deploy-ready SFDX package (object/field metadata, the Flow, deploy scripts, agent runbooks) for repeatable provisioning into a target org.
Value instrumentation	An AI-Generated Documents list view that counts auto-created documents — each row is one request-wait-resubmit cycle eliminated.

Hosting posture (production, to be confirmed). The AIE target is a COTS, commercial-cloud capability at FedRAMP+/IL4. The demonstration ran in a standard commercial trial org, which is not IL4-authorized. A production deployment would target a FedRAMP/IL4-authorized Salesforce environment and an in-boundary model service; the exact authorization status and model hosting should be verified against current AIE requirements before any handling of real applicant data.

6. Operating modes

The same outcome is reachable three ways, which is what makes the capability portable and the demonstration reliable:

- **Native Flow** — synchronous, in-platform creation of the first predicted document on save. Always available; no external compute.
- **Headless API agents** — the Python orchestrator runs the full roster on a schedule against new cadets via the REST API; the production pattern for scale.
- **Supervised browser agent** — performs the identical steps through the UI where API credentials are unavailable or where a live, in-UI demonstration is desired.

Because the agents are idempotent, these modes compose safely: the native Flow may seed a document, and the DocumentAgent then completes the remainder without duplication.

7. Role in the AIE accessions process

The agents map to specific stages of the recruiting and accessions lifecycle, each traceable to an AIE Functional RTM requirement. The medical/waiver stage is the demonstrated bottleneck; the remaining stages are addressed by the same model and are delivered natively in a licensed org.

#	Lifecycle stage	Agent role / automation	RTM	Layer
1	Lead & application	Guided intake pre-fills the case (capture once, reuse everywhere)	2.1.4	Licensed
2	Pre-screen & eligibility	Automated eligibility rules + suitability assist before pipeline entry	2.3.1.6 / 3.8.1.1	Licensed
3	Medical / waiver (bottleneck)	Native Flow creates the first predicted document on save	3.8 (RPA)	Native (live)
4	Full checklist (any factor)	DocumentAgent completes the policy-driven document set	1.3 / 3.3	Agent (live)
5	Case copilot	CopilotAgent writes case summary, gap check, drafted request	3.10	Agent (live)
6	Recruiter worklist	TaskAgent queues the high-priority follow-up task	3.8	Agent
7	DoDMERB / SF-86	Status sync + SF-86 validation; protect the 60-day SECRET SLA	3.3.1.7 / 3.4.4.18-.25	Licensed
8	Selection boards	Packet assembly + completeness scoring (human decides)	2.1.4.7	Licensed
9	Contracting & onboarding	DocuSign generate-and-sign; orchestration	1.3 / 3.4.4	Licensed
10	Commission / accession	Eligibility validation, OML, accession orders	4.7	Licensed

Across the lifecycle the agents act as a tireless documentation coordinator: they ask for the government's next request up front, keep the case complete and current, and let recruiters and adjudicators spend their time on judgment rather than paperwork chase.

8. Governance, guardrails, and compliance

The agents are designed to operate inside accessions policy and SEAD 4. The governing principle is simple: **AI surfaces, drafts, and automates; humans approve every send and own every eligibility, suitability, selection, and clearance decision.**

- **Human-in-the-loop.** No applicant message is sent by an agent; the Copilot produces a draft that a human reviews and approves. No agent renders an eligibility, suitability, or clearance determination.
- **Idempotency and bounded action.** Agents create only what is missing and never duplicate or take destructive action without explicit operator instruction; a dry-run mode previews all writes.

- **Auditability.** Every agent action is a Salesforce record write (document, note, task) on the timeline, providing a complete activity trail; notes are explicitly labeled as external-agent / draft output.
- **Data protection.** Production handling targets IL4 hosting (RTM 1.11.1.59) and governed PII/PHI handling (RTM 1.11.1.75); the model boundary must be an authorized, in-boundary service before any real data is processed.
- **Transparent policy.** Eligibility/document rules live in a visible policy table, not an opaque model — adaptations are reviewed configuration changes.

9. Demonstrated results

The architecture was proven live in a commercial Salesforce trial org. Saving a cadet with a qualifying medical factor automatically created the first predicted document; the external agent completed the full checklist, wrote the AI case summary as a draft note, and the value meter (auto-generated documents) incremented in real time as additional cadets were created. The build paused for human confirmation before each meaningful org change, and the data model, Flow, and agents were captured into a redeployable package.

Illustrative value framing. Each auto-generated document corresponds to a request-wait-resubmit cycle eliminated; planning figures used in briefings (for example, a few round-trip days saved per document) are illustrative and should be replaced with measured program metrics once available.

10. Path to a fully licensed Agentforce / Einstein deployment

Moving from the demonstrated hybrid to a fully native deployment is a substitution, not a rebuild. The Cadet Case data model, the policy table, the record-triggered Flow, and the orchestration sequence remain. The external CopilotAgent is replaced by Einstein prompt templates and an Agentforce assistant operating within the platform's governed model boundary; the headless orchestration can continue to run on a schedule for scale, or be expressed as native scheduled/triggered automation. Because the external layer always produced the same records a licensed org would, the migration carries low functional risk.

11. Assumptions and caveats

- The demonstration used a commercial trial edition; Einstein Prompt Builder and the custom Agentforce agent builder were not licensed in that edition, which is why the external agent layer stands in for them.
- All cadet data is fictional; no real applicant PII was used. The IL4/FedRAMP posture, model boundary, and integration interfaces (MuleSoft, DocuSign, DoDMERB, DCSA) are target-state and must be confirmed against current AIE requirements and authorizations.
- RTM identifiers are drawn from the AIE Functional RTM provided for this effort and are used for traceability; mappings should be validated with the program office.
- Cycle-time and day-savings figures are illustrative planning values, not measured outcomes.

Appendix A — Medical factor to document policy table

This table is the single source of truth the agents act on. Adding a factor or a required document is a row edit, propagated to the DocumentAgent (and, for the native path, a Flow decision branch).

Medical factor	Required documents
ADHD / anxiety medication history	Pharmacy records (7-yr history); prescribing provider notes & treatment dates; statement of current functioning (med-free period); school accommodation records (if any)
Asthma after age 13	Pulmonary function test (PFT); provider treatment history; methacholine challenge test (if required)
Prior orthopedic injury or surgery	Operative report; imaging (X-ray/MRI); physical therapy notes; range-of-motion exam
Vision correction (LASIK/PRK)	Surgical operative report; pre/post-op visual acuity; surgeon clearance letter

Appendix B — Representative agent logic (pseudocode)

The DocumentAgent's idempotent completion loop, simplified:

```
for cadet in cadets_with_factor:

    required = POLICY_TABLE[cadet.medical_factor]
    existing  = documents_on(cadet)
    for doc in required - existing:        # only what's missing
        create_document_request(cadet, doc, predicted=True, status='Requested')
    note = llm_summary(cadet)              # CopilotAgent (Einstein replacement)
    attach_note(cadet, note)               # draft only; human approves send
    create_task(cadet, 'Send document request', priority='High')  # TaskAgent
```

Guardrail: every applicant send is human-approved; agents make no eligibility, suitability, or clearance determination (SEAD 4).